Week 2, Solutions

Statistical Thinking (ETC2420 / ETC5242)

PUBLISHED
Semester 2, 2025

1.

```
dbinom(4, 8, 0.5)                    # (a)
```

```
[1] 0.2734375
```

```
pbinom(15, 28, 0.5)                  # (b)
```

```
[1] 0.7142059
```

```
1 - pbinom(12, 18, 0.5)              # (c)
```

```
[1] 0.04812622
```

```
sum(dbinom(seq(1, 13, by = 2), 14, 0.5)) # (d)
```

```
[1] 0.5
```

2.

```
dpois(3, 5)      # (a)
```

```
[1] 0.1403739
```

```
ppois(2, 5)      # (b)
```

```
[1] 0.124652
```

```
1 - ppois(3, 5)  # (c)
```

```
[1] 0.7349741
```

```
rpois(6, 5)      # (d)
```

```
[1] 6 4 9 3 3 5
```

3.

```
qnorm(0.75)        # (a)
```

```
[1] 0.6744898
```

```
qt(0.3, 10)        # (b)
```

```
[1] -0.541528
```

```
qgamma(0.5, 2, 3)  # (c)
```

```
[1] 0.559449
```

4. Since this is a random sample, all of the variables are iid. Therefore they all have the same mean and variance.

    a. $\text{sd}(X_2) = \text{sd}(X_4) = \sqrt{\text{var}(X_4)} = \sqrt{4} = 2.$
    b. By independence, $\text{var}(X_7 + X_8) = \text{var}(X_7) + \text{var}(X_8) = \text{var}(X_4) + \text{var}(X_4) = 8.$
    c. By the Central Limit Theorem, $\bar{X} \approx \text{N}\left(\mathbb{E}(X_1), \frac{\text{var}(X_1)}{9}\right) = \text{N}\left(7, \frac{4}{9}\right).$

5.

```
x <- rnorm(100000, 1, sqrt(2))
mean(x^2)
```

```
[1] 2.983015
```

6.

```
N <- 5000
x1 <- rnorm(N)    # (a)
x2 <- rexp(N)     # (b)
x3 <- rpois(N, 1) # (c)
x4 <- rcauchy(N)  # (d)
mean(x1)
```

```
[1] 0.0002681252
```

```
mean(x2)
```

```
[1] 1.010668
```

```
mean(x3)
```

```
[1] 0.9792
```

```
mean(x4)
```

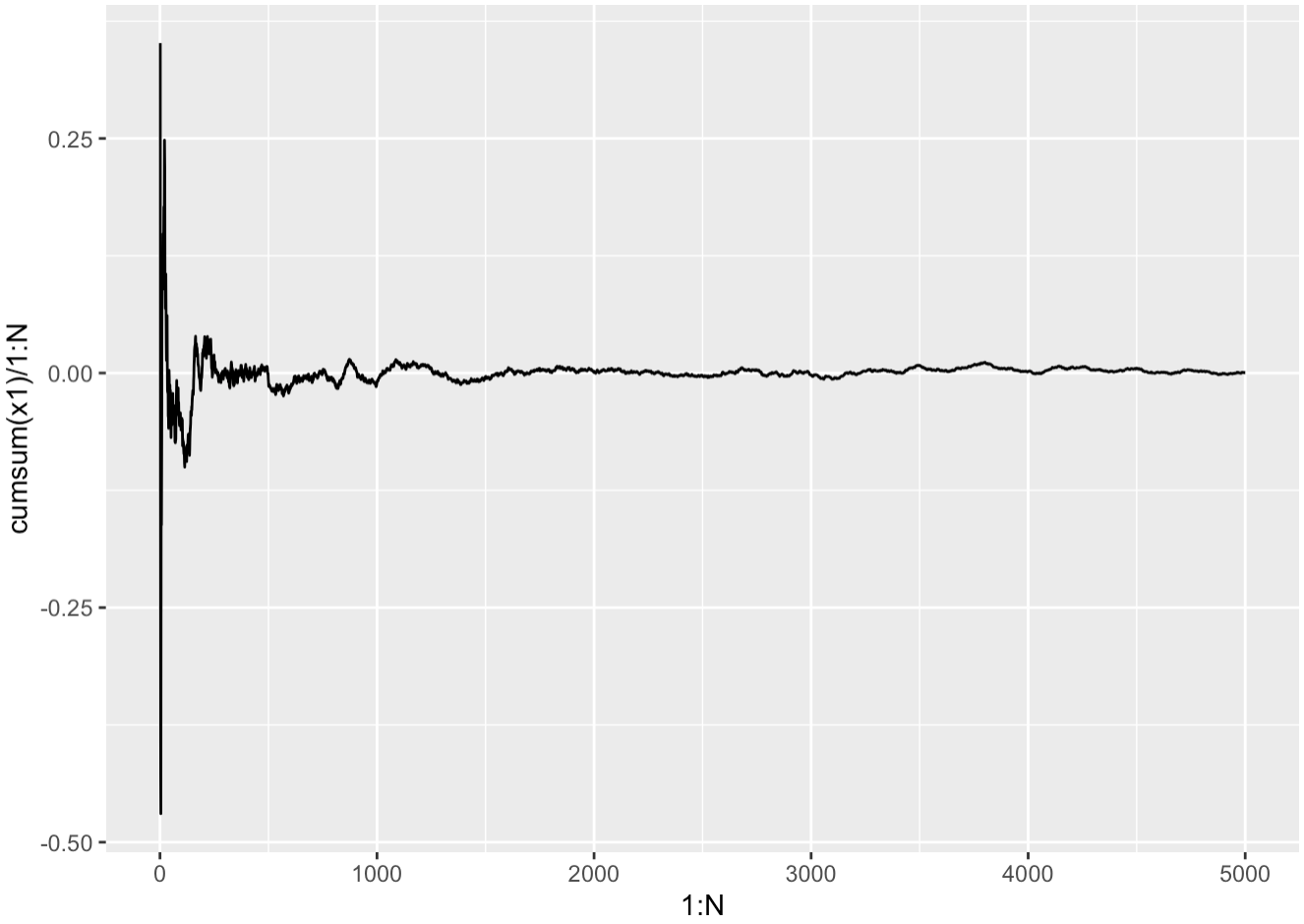
```
mean(x4)
```

```
[1] 2.714852
```

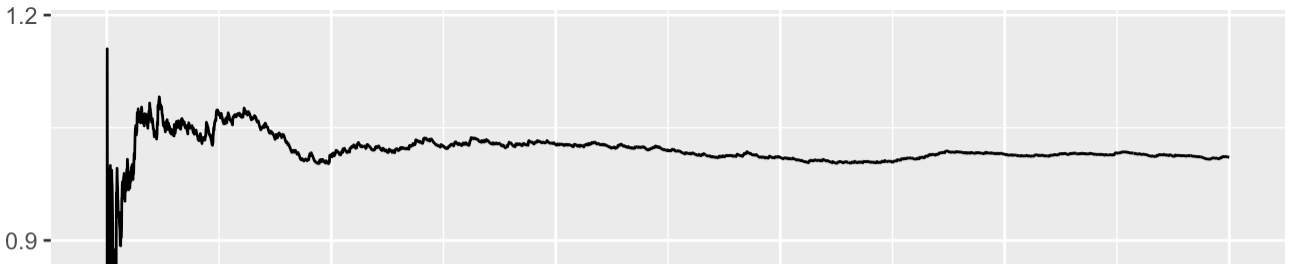
The sample mean for the first three distributions is close to the population mean. The Cauchy distribution doesn't actually have a population mean, although it is unimodal and symmetric around 0. However, the sample mean is not close to zero at all.

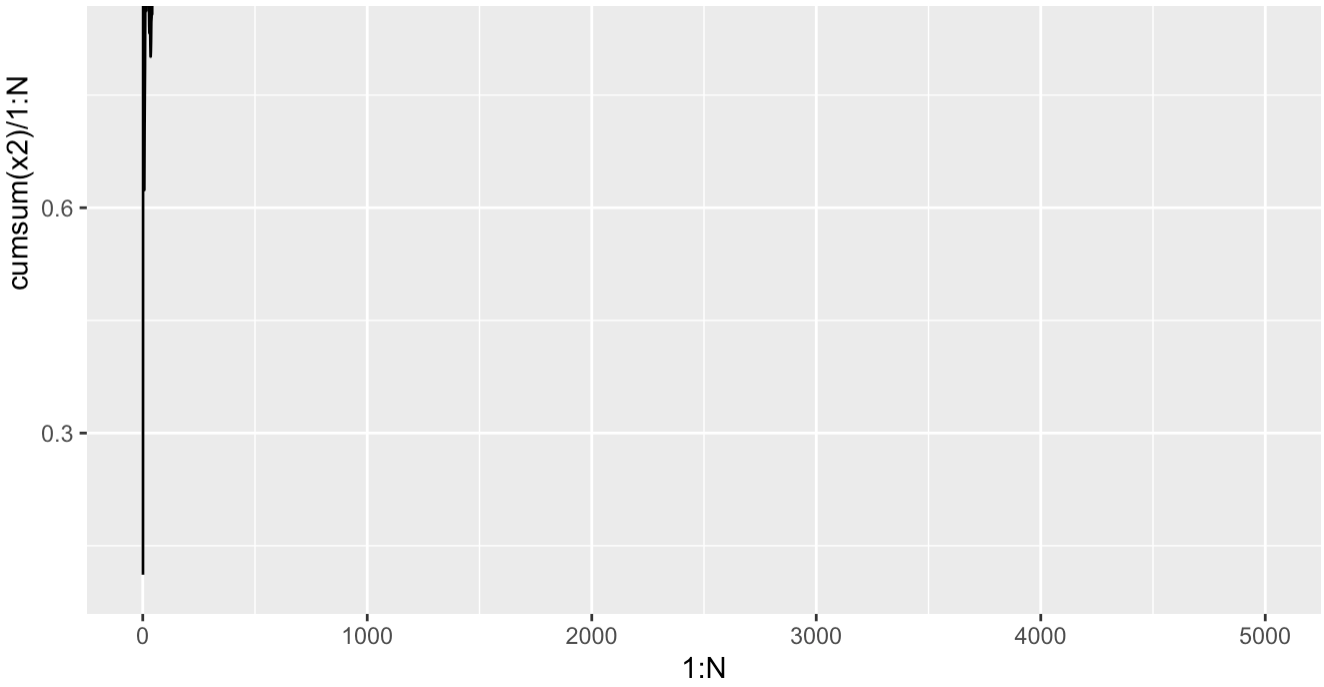
Some more insight can be gleaned by looking at how the sample mean changes as we increase the sample size:

```
ggplot() + aes(x = 1:N, y = cumsum(x1) / 1:N) + geom_line()
```

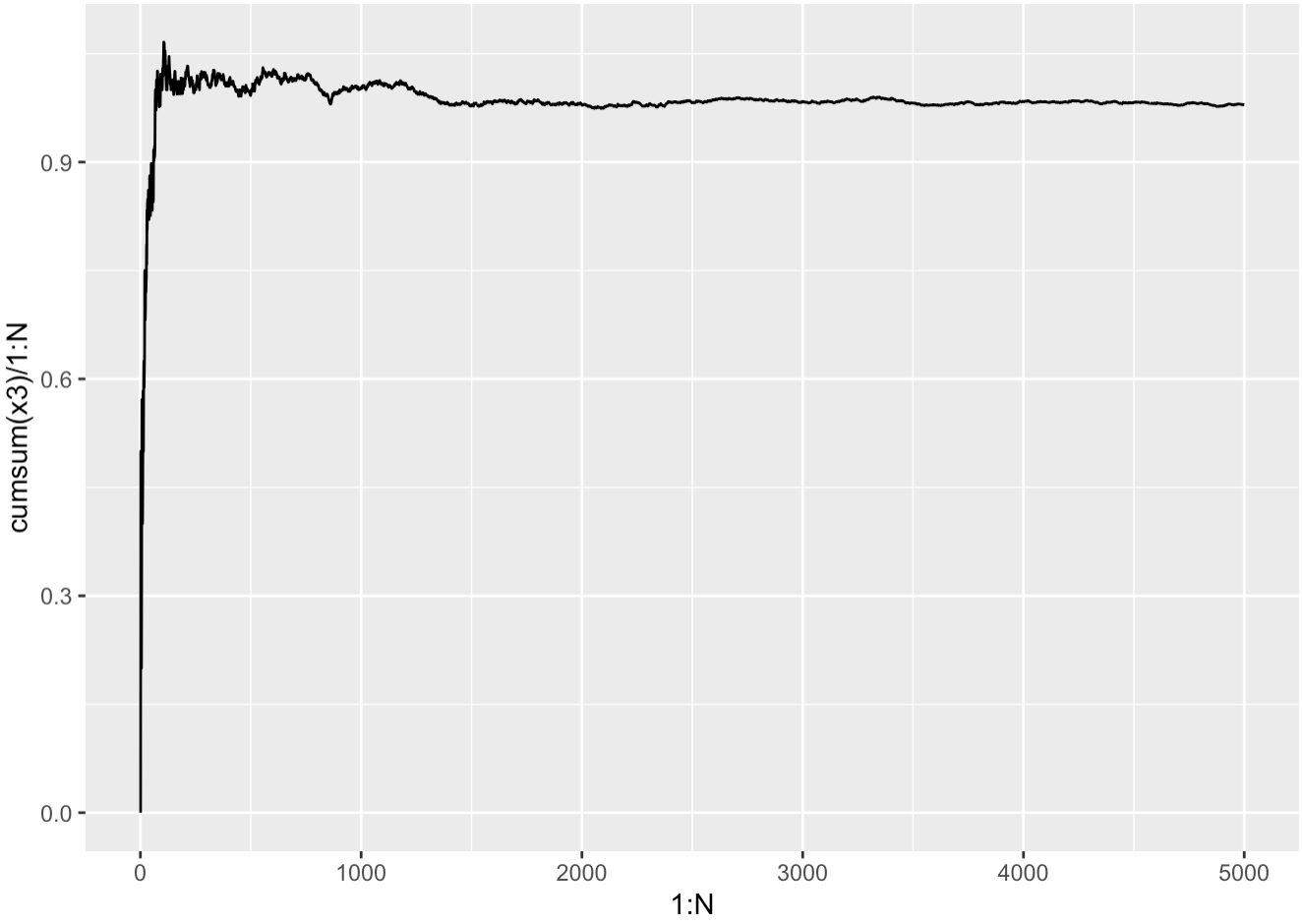



```
ggplot() + aes(x = 1:N, y = cumsum(x2) / 1:N) + geom_line()
```

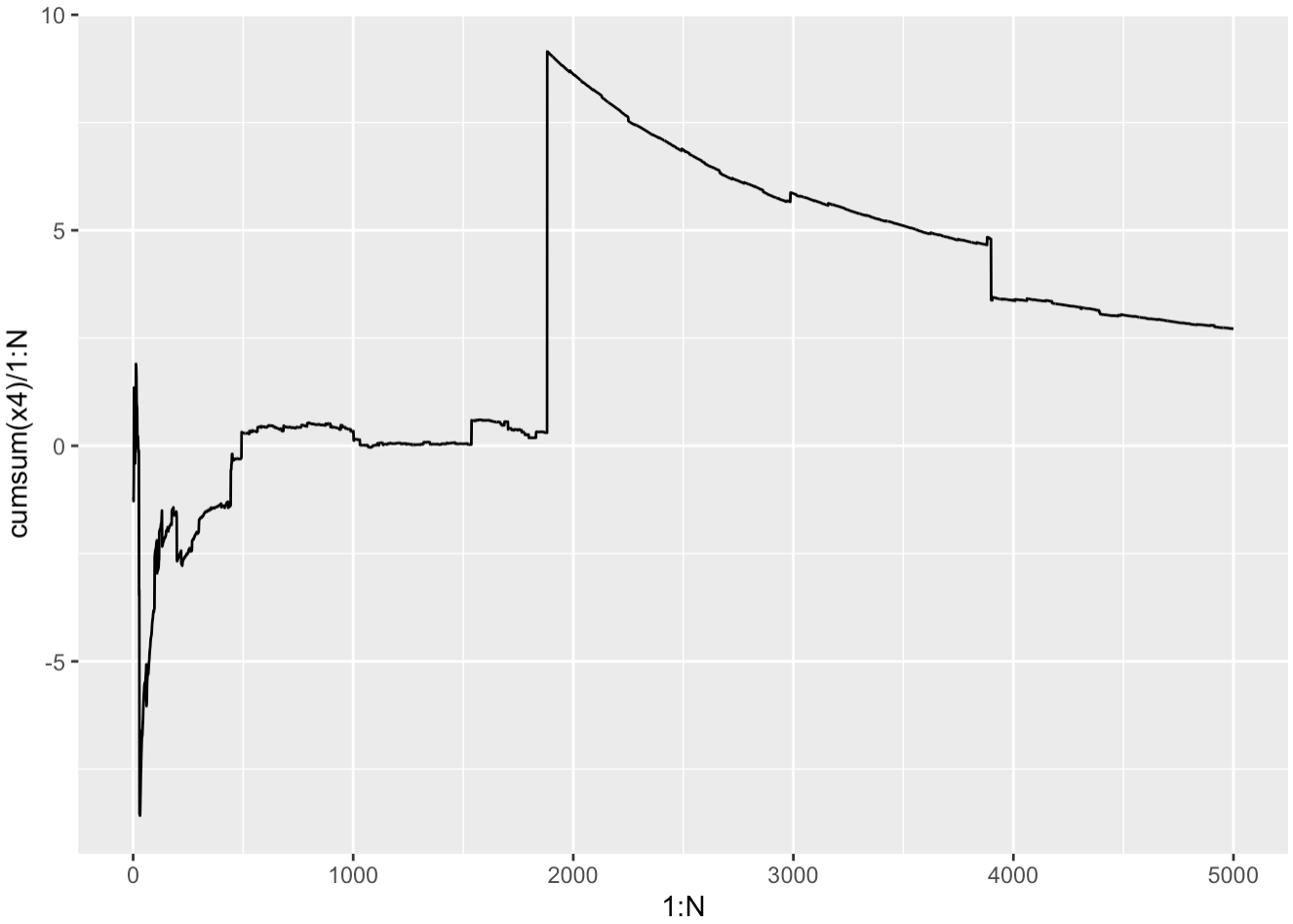





```
ggplot() + aes(x = 1:N, y = cumsum(x3) / 1:N) + geom_line()
```



```
ggplot() + aes(x = 1:N, y = cumsum(x4) / 1:N) + geom_line()
```



The first 3 are converging nicely, but for the the Cauchy distribution the sample mean jumps around erratically.

The LLN doesn't actually work for the Cauchy distribution because it doesn't satisify one of the requirements of the law, namely that the distribution have a finite mean.

7.

```
num_correct <- replicate(5000, sum(sample(1:20) == 1:20))
mean(num_correct)
```

```
[1] 0.9918
```

8.

```
N <- 5000
part1 <- runif(N, min = 0, max = 1)
part2 <- 1 - part1
smaller_part <- ifelse(part1 < part2, part1, part2)
mean(smaller_part)
```

```
[1] 0.2495844
```

The `ifelse()` function is a vectorised version of an `if` statement, which we are using here to demonstrate how it can lead to very compact code. You can write an equivalent solution to the above using `if` instead.

9.

This is a simple generalisation of the problem given as an example. Let's write code that can generalise it to a streak of heads of any length.

```
flips_simulation <- function(streak_length) {
  # Initialise counts.
  num_flips <- 0
  num_heads_in_row <- 0

  # Keep flipping coins until you get the streak of desired length.
  while (num_heads_in_row < streak_length) {

    # Flip a coin.
    new_flip  <- sample(c("H", "T"), size = 1)
    num_flips <- num_flips + 1

    # Increment the counts, depending on what you flipped.
    if (new_flip == "H") {
        num_heads_in_row <- num_heads_in_row + 1
      } else {
        num_heads_in_row <- 0
      }
  }

  return(num_flips)
}

num_flips <- replicate(5000, flips_simulation(5))
mean(num_flips)
```

```
[1] 61.489
```

10.

```
game_simulation <- function() {
  # First roll
  die_roll <- sample(1:6, size = 1)
  score <- die_roll

  # Keep rolling if you get a 6.
  while (die_roll == 6) {
    score <- score - 1  # amend previous added score to be 5 instead of 6
```

```
    die_roll <- sample(1:6, size = 1)
    score <- score + die_roll
  }

  return(score)
}

scores <- replicate(5000, game_simulation())
mean(scores)
```

```
[1] 4.0038
```

11.

First let's wrap up our previous code (from the lecture) into a function:

```
birthday_prob <- function(size = 30, N = 5000) {
  mean(replicate(N, any(duplicated(sample(1:365, size = size, replace = TRUE)))))
}
```

Now we can try it out on groups of varying sizes:

```
birthday_prob(20)
```

```
[1] 0.4146
```

```
birthday_prob(21)
```

```
[1] 0.4478
```

```
birthday_prob(22)
```

```
[1] 0.4708
```

```
birthday_prob(23)
```

```
[1] 0.5116
```

```
birthday_prob(24)
```

```
[1] 0.5362
```

```
birthday_prob(25)
```

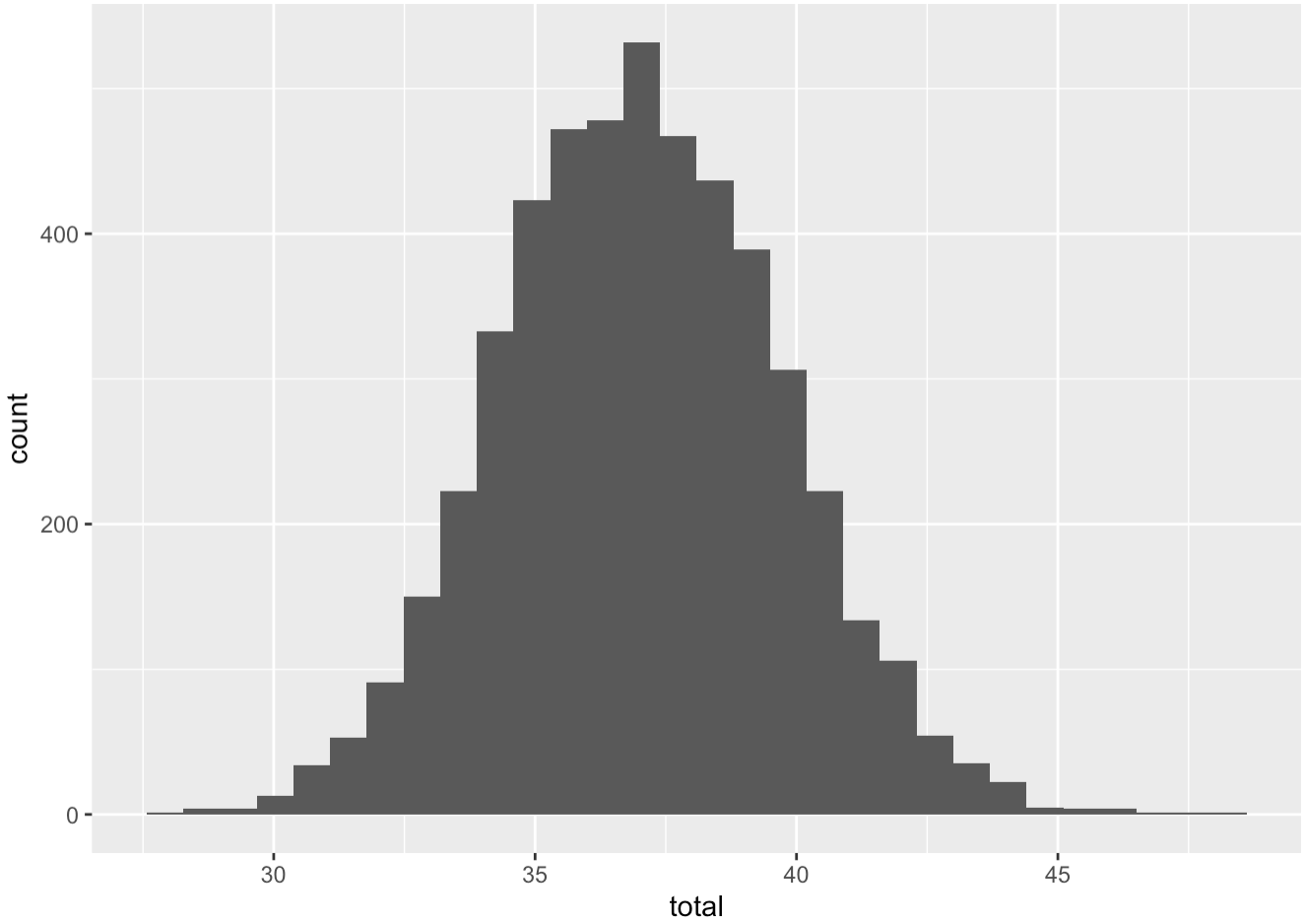
```
[1] 0.5732
```

The answer to the problem is **23 students**. Surprisingly few!

This is a famous problem known as the birthday paradox.

12.

```
N <- 5000
tram  <- rgamma(N, shape =  40, rate = 4)
train <- rgamma(N, shape = 120, rate = 6)
bus   <- rgamma(N, shape =  35, rate = 5)
total <- tram + train + bus
ggplot() + aes(x = total) + geom_histogram()
```




```
mean(total)
```

```
[1] 37.0262
```

```
sd(total)
```

```
[1] 2.67527
```

```
mean(total > 40)
```

```
[1] 0.135
```

13.

```
sort(sample(1:500, size = 20))
```

```
 [1]   6  12  38  54  56  75  98 121 132 166 180 190 224 241 267 298 362 374 399
[20] 498
```

Sorting the values in order isn't strictly necessary, but would be more convenient to have for actual use in practice.

14.

```
student_sample <- sample(   1:5000, size = 15)
staff_sample   <- sample(9001:9200, size = 15)
combined_sample <- sort(c(student_sample, staff_sample))
combined_sample
```

```
 [1]   26  128  472  759 1089 1466 1615 1738 3057 3585 3680 4037 4058 4213 4345
[16] 9002 9020 9027 9031 9041 9050 9064 9067 9076 9079 9094 9120 9122 9136 9151
```

15. a. Number the participants 1 to 40. The green cans can be assigned to the following participants:

```
sort(sample(1:40, size = 20))
```

```
[1]  2  3  4  5  6  9 10 12 13 14 15 19 21 26 30 31 32 34 37 40
```

b. Without randomising the allocation of cans, we won't know for sure whether any differences between the cans are due only to the colour or some other confounding variable. For example, suppose younger participants prefer green cans, and are also willing to pay more for the drink irrespective of the colour. We might mistakenly infer that using green cans is more profitable. Thus, without the randomisation the experiment is substantially less useful.